

debugger pauses to let you inspect the variables and lets you resume stepping through individual lines of code. In most IDEs, you can set a breakpoint by double-clicking the line numbers on the left side of the window.

The values currently stored in the program's variables are displayed somewhere in the debugging window in every debugger. However, one common method of debugging your programs is *print debugging*: adding `print()` calls to display the values of variables and then rerunning your program. While simple and convenient, this approach to debugging can often be slower than using a debugger. With print debugging, you must add the `print()` calls, rerun your program, and then remove the `print()` calls later. However, after rerunning the program, you'll often find that you need to add more `print()` calls to see the values of other variables. This means you need to rerun the program yet again, and this run could reveal that you need another round of adding `print()` calls, and so on. Also, it's common to forget some of the `print()` calls you've added, requiring an additional round of deleting `print()` calls. Print debugging is straightforward for simple bugs, but using an actual debugger can save you time in the long run.

Summary

Programming is a fun, creative skill to develop. Whether you've mastered the basics of Python syntax or simply want to dive into some real Python programs, the projects in this book will spark new ideas for what's possible with as little as a few pages of code.

The best way to work through these programs isn't to merely read their code or copy and paste it to your computer. Take the time to manually type the code from this book into your editor to develop the muscle memory of writing code. This also slows you down so you can consider each line as you type, instead of merely skimming it over with your eyes. Look up any instructions you don't recognize using an internet search engine, or experiment with them in the interactive shell.

Finally, take it upon yourself to re-create the program from scratch and then modify it with features of your own. These exercises give you a solid foundation for how programming concepts are applied to create actual, runnable programs. And most of all, don't forget to have fun!